

TypedArray Find Within

Subsequence search for TypedArrays

Proposing advancement to **Stage 2**

James M Snell

Recap: The Problem

TypedArrays have `indexOf` for single elements, but no way to search for a subsequence.

```
// The current approach – manual linear search
function findSubsequence(haystack, needle) {
  if (needle.length === 0) return 0;
  if (needle.length > haystack.length) return -1;
outer: for (let i = 0; i ≤ haystack.length - needle.length; i++) {
  for (let j = 0; j < needle.length; j++) {
    if (haystack[i + j] !== needle[j]) continue outer;
  }
  return i;
}
return -1;
}
```

- No built-in subsequence search for TypedArrays
- Userland implementations cannot use optimized algorithms (Boyer-Moore, etc.)
- Common need in streams, protocol parsing, binary format handling
- Node.js `Buffer.indexOf` supports this, but it is non-standard

The Solution: Three Prototype Methods

Method	Returns	Searches
<code>.search(needle [, position])</code>	Index of first match	Forward from <code>position</code>
<code>.searchLast(needle [, position])</code>	Index of last match	Backward from <code>position</code>
<code>.contains(needle [, position])</code>	<code>true</code> / <code>false</code>	Forward from <code>position</code>

```
const enc = new TextEncoder();
const u8 = enc.encode('Hello TC39, Hello TC39');

u8.search(enc.encode('TC39'));           // 6
u8.search(enc.encode('TC39'), 7);        // 17
u8.searchLast(enc.encode('TC39'));        // 17
u8.searchLast(enc.encode('TC39'), 16);    // 6
u8.contains(enc.encode('TC39'));          // true
u8.contains(enc.encode('TC39'), 18);      // false
```

Needle Types

The `needle` is any iterable object (except `String`). Elements are type-checked against the haystack.

```
const u8 = new Uint8Array([1, 2, 3, 4, 5]);

u8.search(new Uint8Array([3, 4])); // 2 - same-type TypedArray
u8.search(new Int16Array([3, 4])); // 2 - different-type TypedArray (iterated)
u8.search([3, 4]);                 // 2 - plain Array (iterable)
u8.search('hello');                // TypeError - strings rejected
u8.search(42);                     // TypeError - non-iterable rejected
```

Type checking: If the iterable yields values of the wrong type (e.g., `BigInt` for a non-`BigInt` haystack), the search returns `-1` rather than throwing – the needle can never match.

Snapshot semantics: The needle is always collected into a `List` via `@@iterator` before the search begins, ensuring correctness when the needle is backed by a `SharedArrayBuffer`.

Spec Text Status

Full spec text covers all three methods plus supporting abstract operations.

Abstract Operations:

- `ToCompatibleTypedArrayElementList` — validates and snapshots the needle
- `TypedArraySearchSubsequence` — the core search (shared by all three methods)
- `SequenceSameValueZeroEqual` — element-wise comparison
- `ValidateIntegralNumber` — validates the position argument

Implementation freedom is explicit in the spec:

"Implementations may use any technique to search for the subsequence, such as naive search, Boyer-Moore, or other matching algorithms, provided the observable result is correct."

Design Decisions Since Stage 1

Feedback from the November 2025 plenary and subsequent discussion:

1. `searchLast` added — reverse search support (#2)
2. `position` parameter on all three methods (#4)
3. Needle accepts any iterable, not just same-type TypedArrays (#1)
4. String needles throw `TypeError` — code point iteration is unlikely to be intended
5. Needle is snapshotted via `@@iterator` — handles `SharedArrayBuffer`-backed needles correctly (#8)
6. Search algorithm is implementation-defined — spec defines the expected result, not the technique (#3)
7. Wrong-type needle elements return `-1` (current spec) rather than throwing (#7)

Open Issues

Open: Is `contains` Justified?

#6 — @michaelficarra

Why not just `search(needle)  $\equiv$  -1`?

Arguments for keeping `contains`:

- Mirrors `indexOf` / `includes` pattern already in the language
- Clearer intent — boolean result is self-documenting
- Potential for implementation to short-circuit (no need to track index)

Arguments for dropping it:

- `search(needle)  $\equiv$  -1` is trivial and well-understood
- Reduces API surface area
- `indexOf` / `includes` exist as separate methods, but they predate each other

Current spec includes `contains` with an editor's note flagging this as open. Seeking committee guidance.

Open: Throw or Return `-1` for Wrong Types?

#7 — @jasnell

When the needle contains wrong-type elements, should we throw or return `-1` / `false` ?

```
const u8 = new Uint8Array([1, 2, 3]);  
// Needle yields BigInts – can never match a Uint8Array  
u8.search(new BigInt64Array([1n, 2n])); // -1 or TypeError?
```

	Return <code>-1</code>	Throw <code>TypeError</code>
Rationale	Values can never match, so <code>-1</code> is correct	Wrong types are likely a bug
Precedent	<code>indexOf</code> returns <code>-1</code> for non-matching types	Stricter APIs throw early
Current spec	Returns <code>-1</code> (via <code>~not-found~</code>)	Alternative noted in spec

Seeking committee preference. This also affects `contains` (return `false` vs throw).

Open: SharedArrayBuffer-backed Needles

#8 — @bakkot

How should TypedArray needles be handled — especially when backed by a SharedArrayBuffer?. Currently snapshots needle via `@@iterator` before search, prevents mutating a SAB-backed needle mid-search.

```
// Worker could change needle from [2,3] to [3,2] mid-search
let needle = new Uint8Array(sharedBuffer);
Uint8Array.of(2, 2, 3, 3, 2).search(needle); // could return -1 without snapshot
```

Strategy	Copies needle
Always snapshot (current spec)	Always
Snapshot if shared	Only SAB-backed
Never snapshot	Never (user syncs SAB)

The haystack is never snapshotted (`indexOf` / `lastIndexOf` precedent). Seeking committee guidance.

Other Open Issues (Addressed in Spec)

Needle type (#1) — @mhofman

- Should needle accept any array-like, not just same-type TypedArrays?
- Addressed: needle accepts any iterable object; type-checked per element

Find from end (#2) — @mhofman

- Addressed: `searchLast` method added

Implementation-defined algorithm (#3) — @jasnell

- Addressed: spec defines expected result; algorithm is implementation-defined

Find from offset (#4) — @bakkot

- Addressed: `position` parameter on all three methods

Summary

Issue	Status	Seeking
Is <code>contains</code> justified?	Open	Committee guidance
Throw vs <code>-1</code> for wrong types	Open	Committee preference
SAB-backed needle: snapshot always / if shared / never	Open	Committee input
Needle type (accept iterables)	Addressed in spec	–
Find from end (<code>searchLast</code>)	Addressed in spec	–
Implementation-defined algorithm	Addressed in spec	–
Find from offset (<code>position</code>)	Addressed in spec	–

Asking for Stage 2

Stage 2 entrance criteria:

- Initial spec text covering all major semantics
- Committee expectation that the feature will be developed and eventually included
- Identified open issues are appropriate for Stage 2 resolution

What we have:

- Complete spec text for `search`, `searchLast`, and `contains`
- Shared abstract operations (`ToCompatibleTypedArrayElementList`, `TypedArraySearchSubsequence`)
- Explicit affordance for implementation-defined search algorithms
- Clear set of open questions suitable for stage 2 refinement

Requesting advancement to Stage 2.